

Name : Saswata Kumar Nayak

Mail id : 22052504@kiit.ac.in

Superset id : 6363187

University Roll : 22052504

Week - 07 (More on React)

9. Create a React CricketApp : -

Objectives

- List the features of ES6
- Explain JavaScript let
- Identify the differences between var and let
- Explain JavaScript const
- Explain ES6 class fundamentals
- Explain ES6 class inheritance
- Define ES6 arrow functions
- Identify set(), map()

1. ES6 Features

let, const, arrow functions, classes, inheritance, template literals, destructuring, spread/rest, map, set, promises, modules.

2. let

Block-scoped variable. Can be updated, not re-declared.

3. var vs let

var: function-scoped, can re-declare.

let: block-scoped, no re-declare.

4. const

Block-scoped constant. No re-assignment allowed.

5. Classes

Use `class` to define objects with constructor and methods.

6. Inheritance

Use `extends` and `super()` to inherit properties/methods.

7. Arrow Functions

Short syntax, no own `this`.

Example: `(a, b) => a + b`

8. Set & Map

Set: Stores unique values.

Map: Key-value pairs with any data type as key.

Output:



List of Players

- Mr. Ishan Score: 95
- Mr. Ashwin Score: 55
- Mr. Virat Score: 45
- Mr. Jadeja Score: 75
- Mr. Rohit Score: 30
- Mr. Gill Score: 78
- Mr. Bumrah Score: 39
- Mr. Surya Score: 40
- Mr. Dhoni Score: 72
- Mr. Shami Score: 33
- Mr. Rishabh Score: 63

List of Players having Scores Less than 70

- Ashwin - 55
- Virat - 45
- Rohit - 30
- Bumrah - 39
- Surya - 40
- Shami - 33
- Rishabh - 63



Indian Team

Odd Players

- First: Virat Kohli
- Third: M.S. Dhoni
- Fifth: Pujara

Even Players

- Second: R.Jadeja
- Fourth: Sarfaraz
- Sixth: Sundar

List of Indian Players Merged:

- Virat Kohli
- R.Jadeja
- M.S. Dhoni
- Sarfaraz
- Pujara
- Sundar

10. Create a React Application officespacerentalapp

Objectives

- Define JSX
- Explain about ECMA Script
- Explain React.createElement()
- Explain how to create React nodes with JSX
- Define how to render JSX to DOM
- Explain how to use JavaScript expressions in JSX
- Explain how to use inline CSS in JSX

1. Define JSX

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code in JavaScript. It is used with React to describe UI elements.

2. Explain about ECMA Script

ECMAScript is the official standard for JavaScript. ES6 (ECMAScript 2015) introduced modern features like let, const, arrow functions, classes, and modules.

3. Explain React.createElement()

This function creates React elements without JSX.

Example:

```
React.createElement('h1', null, 'Hello')
```

4. Create React nodes with JSX

Instead of React.createElement, you can use JSX to create elements.

Example:

```
const heading = <h1>Hello</h1>;
```

5. Render JSX to DOM

JSX elements are rendered using ReactDOM.

Example:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

6. JavaScript expressions in JSX

You can use JS expressions inside JSX using curly braces.

Example:

```
const name = "Ram";
```

```
<h1>Hello {name}</h1>;
```

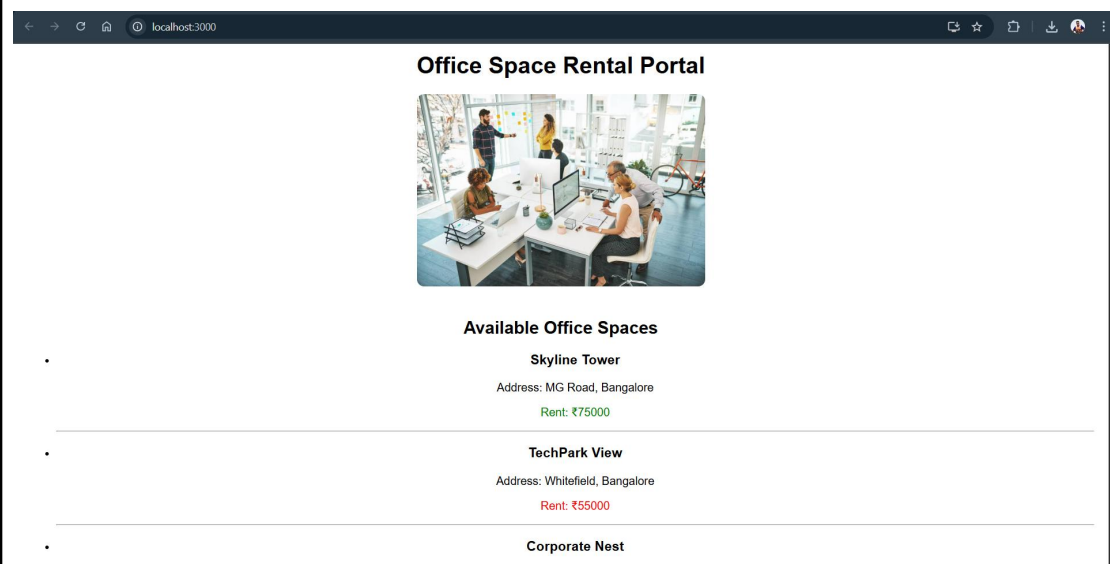
7. Inline CSS in JSX

Use style as an object with camelCase properties.

Example:

```
<h1 style={{ color: 'blue', fontSize: '20px' }}>Styled Text</h1>
```

Output :



11. Create a React Application eventexamplesapp :

Objectives

- Explain React events
- Explain about event handlers
- Define Synthetic event

Identify React event naming convention

Solution

1. React Events

React events are used to handle user interactions like clicks, input, or form submissions. They are similar to browser events but handled in a React-friendly way using JSX.

2. Event Handlers

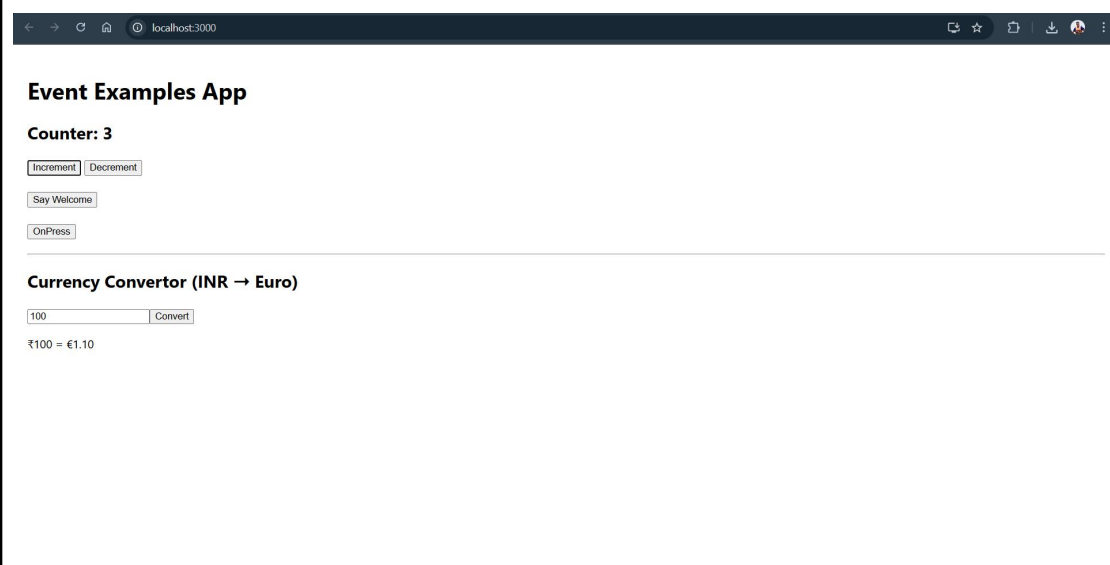
Event handlers are functions triggered when an event occurs, like a button click or a key press. In React, these are passed as props using JSX.

3. Synthetic Event

A SyntheticEvent is React's wrapper around the browser's native event system. It provides a consistent and cross-browser interface to handle events efficiently.

4. React Event Naming Convention

React uses camelCase for event names instead of lowercase. For example, onclick becomes onClick, onchange becomes onChange.



12. Create a React Application named ticketbookingapp :

Objectives

- Explain about conditional rendering in React
- Define element variables
- Explain how to prevent components from rendering

1. Conditional Rendering in React

Conditional rendering is used to show or hide components or elements based on certain conditions. It works like regular JavaScript conditions using if-else, ternary operators, or logical &&.

2. Element Variables

Element variables are used to store JSX elements in a variable. This allows you to decide what to render before the return statement, helping in conditional rendering.

Example:

```
js
CopyEdit
let message;if (isLoggedIn) {
  message = <p>Welcome back!</p>;
} else {
  message = <p>Please sign in.</p>;
}
```

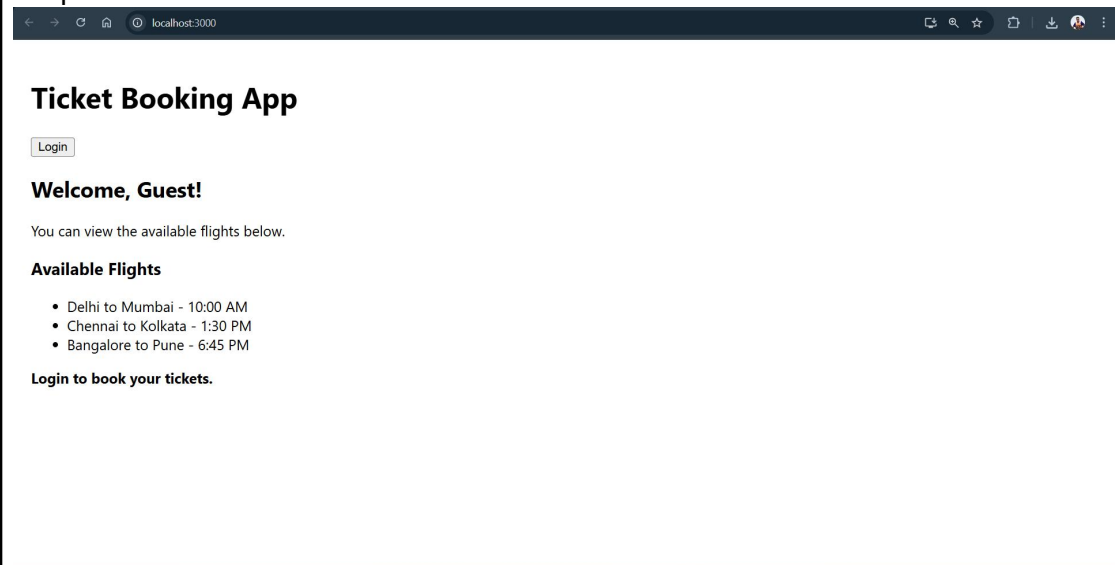
3. Preventing Components from Rendering

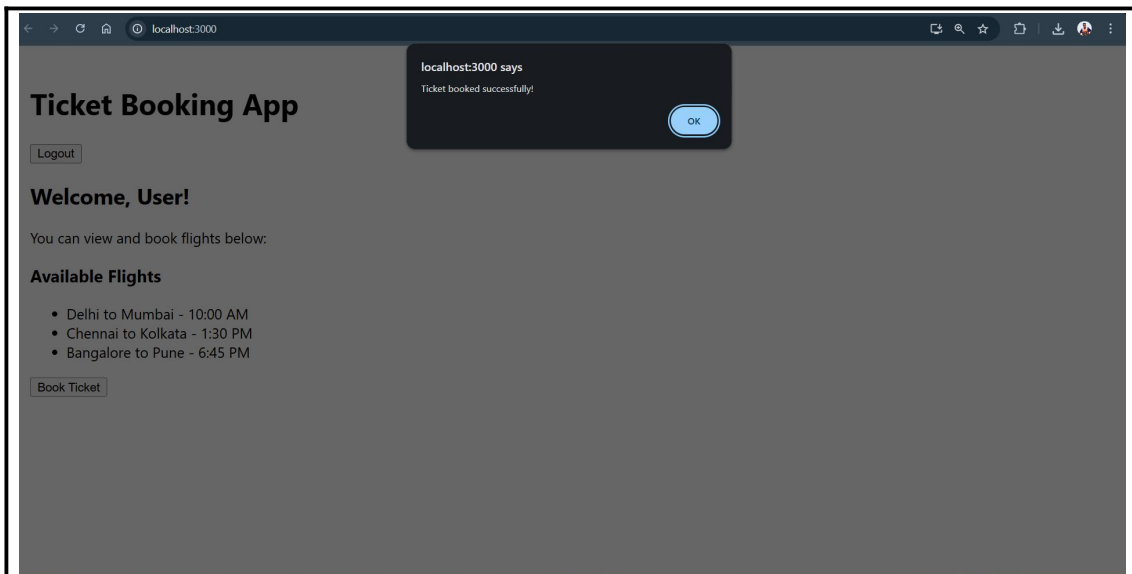
To prevent a component from rendering, you can return null from its render method or component function. Returning null means React will skip rendering that component.

Example:

```
js
CopyEdit
if (!isVisible) {
  return null;
}
```

Output :





13. Create a React App named bloggerapp :

Objectives

- Explain various ways of conditional rendering
- Explain how to render multiple components
- Define list component
- Explain about keys in React applications
- Explain how to extract components with keys
- Explain React Map, map() function

In React, conditional rendering allows components to display different content based on conditions. This can be done in several ways, including using standard if-else statements outside the return block, the ternary operator within JSX for concise conditions, logical AND (&&) for rendering content only when a condition is true, and even switch statements for handling multiple cases. These methods help in dynamically controlling what gets rendered on the screen.

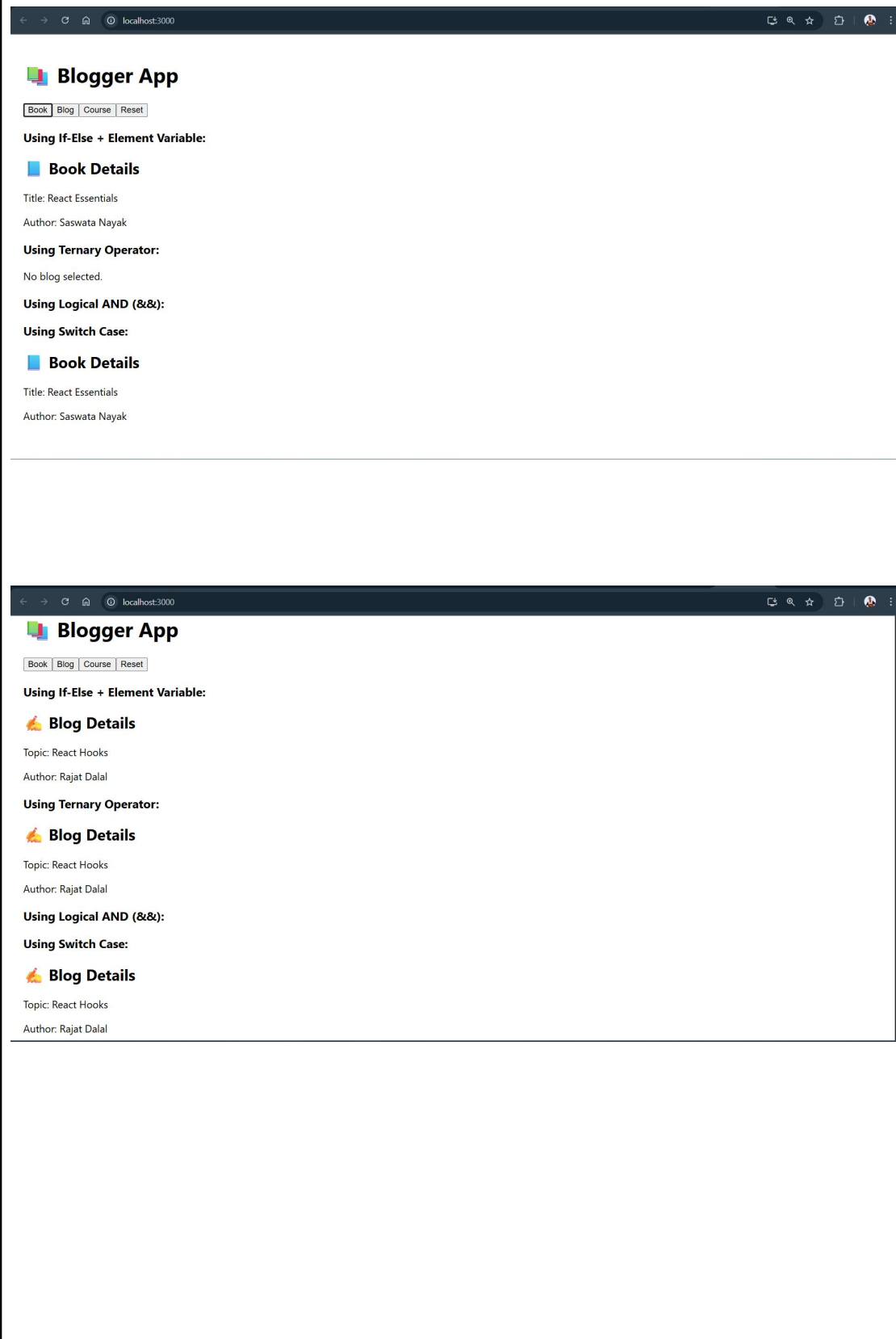
React also supports rendering multiple components together. This is often done using React Fragments (<> </>) to group multiple elements without adding extra nodes to the DOM. Components can be nested or returned as part of an array, making it easy to build complex UIs with multiple parts shown at once.

A list component in React is one that displays a set of items using data from an array. This is commonly done using the map() function, which loops through the array and returns JSX elements for each item. When rendering such lists, it's important to assign a key to each item. Keys are unique identifiers that help React efficiently update or re-render only the changed items instead of the entire list. These keys must be stable and unique, often using an id or index value from the data.

When extracting a child component inside a list, keys should be assigned to the outer JSX element within the list rather than passing them directly to the component as a prop. This ensures React can correctly identify and manage the individual elements in the DOM.

Finally, the `map()` function is a JavaScript array method heavily used in React to render lists. It allows developers to transform an array of data into an array of JSX elements, enabling dynamic and data-driven UI rendering in a simple and declarative way.

Output :



← → ↺ 🏠 🌐 localhost:3000 🔍 ☆ 📄 👤 ⋮

 **Blogger App**

Book

Blog

Course

Reset

Using If-Else + Element Variable:

 **Course Details**

Name: Full Stack Development

Duration: 3 months

Using Ternary Operator:

No blog selected.

Using Logical AND (&&):

 **Course Details**

Name: Full Stack Development

Duration: 3 months

Using Switch Case:

 **Course Details**

Name: Full Stack Development